Report Lab 4

Introduction

In the fourth installment of the computer architecture lab we are asked to implement a set of prefetchers in the ChampSim simulator environment. We are then asked to evaluate their performance in a system with a wide memory bandwidth and a limited memory bandwidth. We further then compare these different implementations to understand the different tradeoffs and effects that choices in the implementation have on the overall performance of the system. In a last step we compare the implemented prefetchers to the stat-of-the-art pythia prefetcher.

Warmup

In the warmup we are asked to run the baseline CPU simulation without a prefetcher attatched. In Figure 1 we can see the IPC plot for the baseline configuration. It is easy to see that there are significant differences between the applications performance. We can clearly see that charlie_0 has the highest ICU and is therefore the most performant or fastest trace, while bc-12 and bfs-14 are the least performant or the slowest. The bc-0 trace is also not far off from bc-12 and bfs-14. Further we observe that the traces from the GAP dataset are a lot slower than the traces from the charlie dataset.

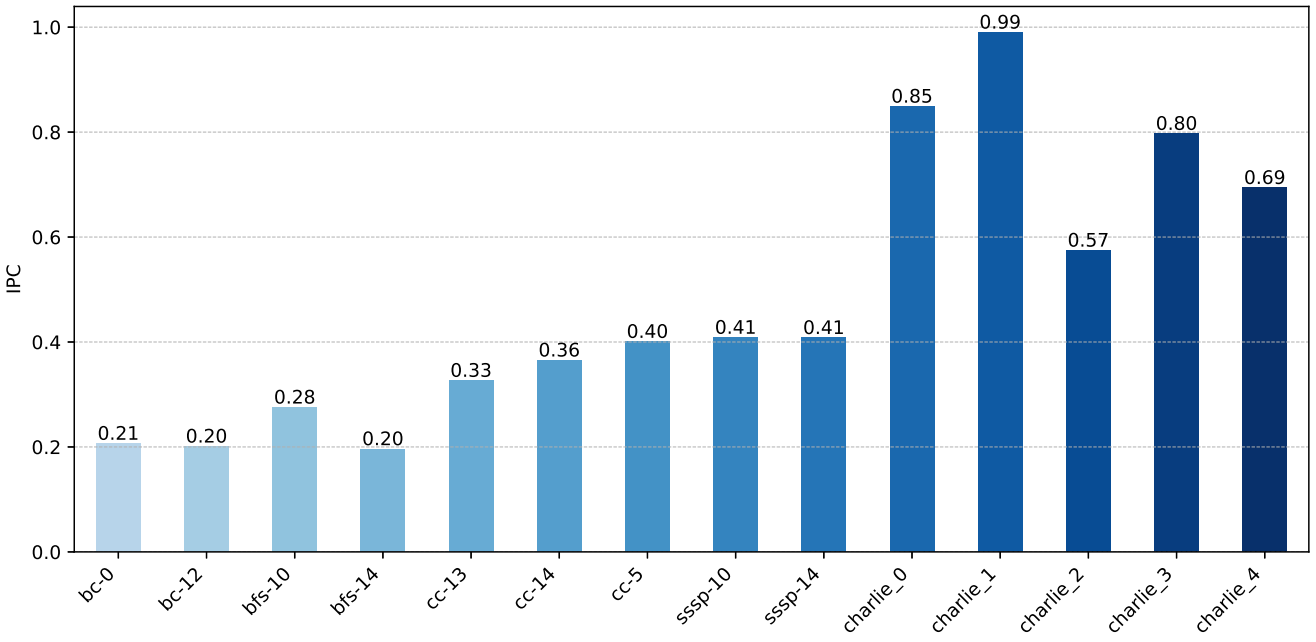

Figure 1: No prefetcher ICU performance.

Task 1

In the first task we are asked to implement a basic global-history-buffer-based stride prefetcher. A description of the algorithm is given in the paper by Kyle et. al.[1]

Implementation

The algorithm makes use of two main structures. A global-history-buffer(GHB) and an index-table(IT).

```
// global-history-buffer entry struct
typedef struct {
    uint64_t gm_addr; // global-miss-address
    uint16_t ghb_link; // link to previous ghb_entry
    uint16_t tag; // index table tag
    uint16_t head; // ghb_pointer that holds the head at the time off adding the entry
} ghb_entry_t;

// index-table entry struct
typedef struct {
    uint16_t ghb_ptr; // pointer to the GHB entry
    uint16_t tag; // tag optained from the PC
    bool valid; // valid bit for the entry
} it_entry_t;
```

On a cache access an index/tag/address triple is created. Where the index refers to the position inside the IT and is made up of the lowest 8 bits of the instruction pointer and the tag of the next higher 8 bits. The address is the block-address of the accessed cache block. The prefetcher probes the IT. If successfull it traverses the GHB to find the two previous memory accesses with the same index and tag. From these it can compute the respective strides in memory. If these match then the prefetcher prefetches the next 6 entries with the same strides. As a last step the prefetcher adds the newest cache access to its IT and GHB.

For this first implementation many values were hardcoded to show the advantage of dynamically changing them during runtime in a later task.

Results

In Figure 2 we see the speedup for the implemented GHB-stride prefetcher relative to the baseline evaluated in the Warmup step of the lab. Charlie_2 and charlie_3 show the smallest speedup with 1.01 but in general it can be observed that the traces from the charlie dataset show barely any performance difference. On the other hand we see very clear improvements for the traces in the GAP dataset. The biggest speedup is seen in bfs-14 which is the trace with the worst baseline ICU. Observe also, that no traces shows a speedup below 1. So adding a prefetcher in this system configuration shows a strict performance increase.

Task 2

In the second task we are asked to evalueate the GHB-stride prefetcher in a system with limited memory bandwidth.

Results

In Figure 3 we can see the speedup of the GHB-stride prefetcher relative to the baseline without a prefetcher, both run in a system with a limited memory bandwidth. We can make a few important observations. We can directly see that the geometric mean of the speedup has decreased indicating that the GHB-stride prefetcher does not only perform worse in absolute numbers with a limited bandwidth but also its relative performance to the baseline decreases. The biggest performance difference can be seen in the traces sssp-10 and ssp-14, with -0.11 and -0.09 respectively. The smallest performance loss can be observed with charlie_1 and charlie_2 which show a difference of -0.01. In general the traces from the charlie dataset show a smaller performance loss than those from the GAP dataset. But for the trace charlie_3 we see a speedup of 0.99, meaning that it runs slower with the prefetcher than without.

[1] Kyle J. Nesbit and James E. Smith. Data cache prefetching using a global history buffer. In HPCA, 2004

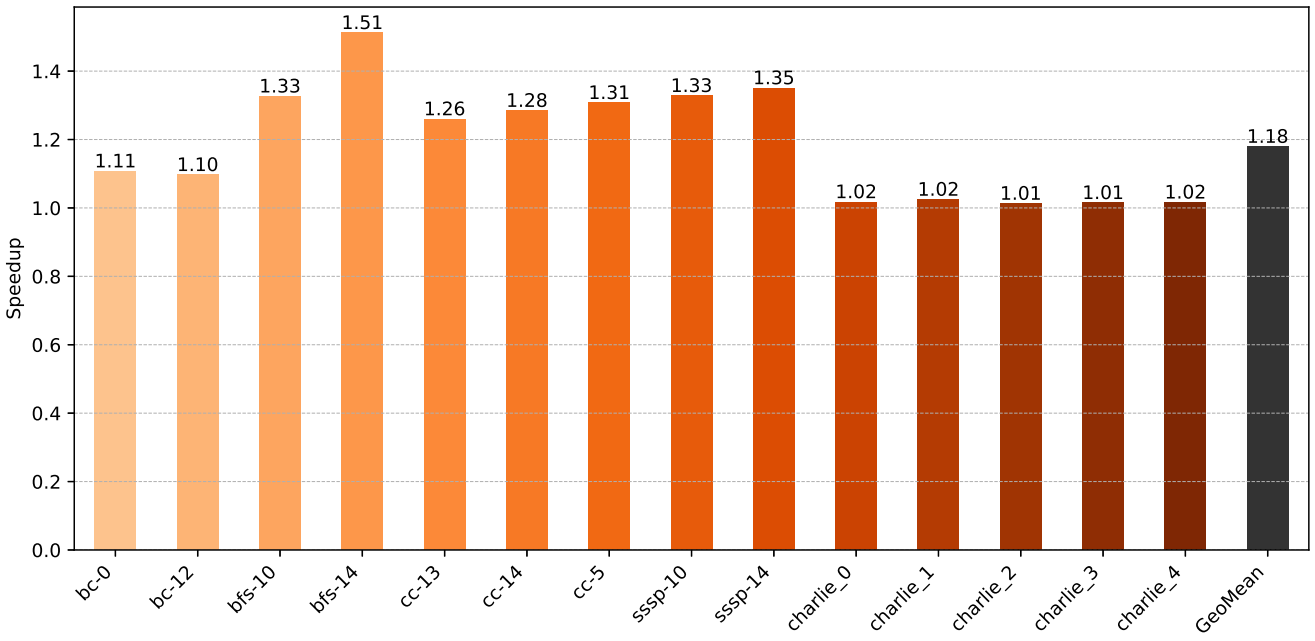

Figure 2: GHB-stride prefetcher speedup at full bandwidth.

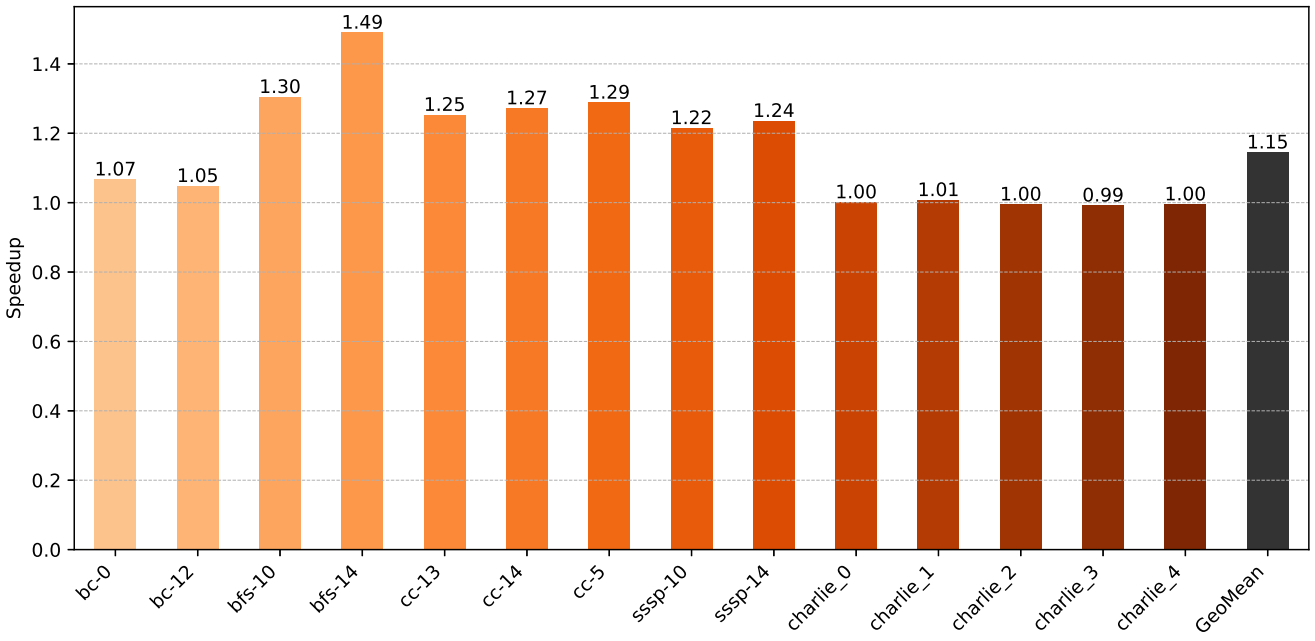

Figure 3: GHB-stride prefetcher speedup at limited bandwidth.

Discussion

In the data we see a performance loss in every trace. This stems from the implementation of the prefetcher. The prefetcher is not aware of the system and therefore does not know how much memory bandwidth is still free. In the implementation we hardcoded the prefetch degree meaning that the prefetcher will always prefetch the next 6 memory addresses. If the memory bus is close to or already at capacity this becomes an issue as it in the best case slows down the prefetching and in the worst case slows down the entire CPU as other memory, that is needed, cannot be fetched due to the bus being clogged by prefetcher requests.

Task 3

In the third task we were asked to extend our prefetcher with system awareness. The system awareness is based on a hueristic scheme to increase or decrease the prefetch degree based on the available memory bandwidth and the accuracy of the prefetcher.

Implementation

The system awareness is represented by the `prefetch_distance` variable in the code. This variable is equivalent to the prefetch degree from the task description. To allow for online changing of this parameter a cycle counter is implemented. Once an epoch has passed (1000 cycles) the performance of the last epoch is evaluated by requesting the memory bandwidth usage and calculating the prefetch efficiency. With this information the prefetcher can then adapt the prefetch degree.

```
// method to calculate the prefetch distance
int16_t ghb_stride_sys_aware::calculate_prefetch_distance(float accuracy, uint8_t memory_bw_usage)
{
  if (memory_bw_usage >= GHB_MEMORY_BW_USAGE_UPPER_THRESHOLD) {
    if (accuracy >= GHB_ACCURACY_UPPER_THRESHOLD)
      return prefetch_distance;
    if (accuracy >= GHB_ACCURACY_LOWER_THRESHOLD)
      return prefetch_distance - 1;
    return prefetch_distance - 2;
  }

  if (memory_bw_usage >= GHB_MEMORY_BW_USAGE_LOWER_THRESHOLD) {
    if (accuracy >= GHB_ACCURACY_UPPER_THRESHOLD)
      return prefetch_distance + 1;
    if (accuracy >= GHB_ACCURACY_LOWER_THRESHOLD)
      return prefetch_distance;
    return prefetch_distance - 1;
  }

  if (accuracy >= GHB_ACCURACY_UPPER_THRESHOLD)
    return prefetch_distance + 2;
  if (accuracy >= GHB_ACCURACY_LOWER_THRESHOLD)
    return prefetch_distance + 1;
  return prefetch_distance;
}
```

Results

Figure 4 shows the speedup of the system-aware GHB-stride prefetcher with full bandwidth relative to the system with no prefetcher at full bandwidth. If we compare this with Figure 2 from Task 1 we see no difference in performance. This makes sense as we initialized our system-aware prefetcher at the hardcoded values for the non-system-aware prefetcher. If the prefetcher is never limited by the memory bandwidth then he will perform the same as the non-system-aware prefetcher.

Figure 5 shows the speedup of the system aware GHB-stride prefetcher with limited bandwidth relative to the system with no prefetcher at limited bandwidth. We see that the geometric mean is higher for the system-aware prefetcher.

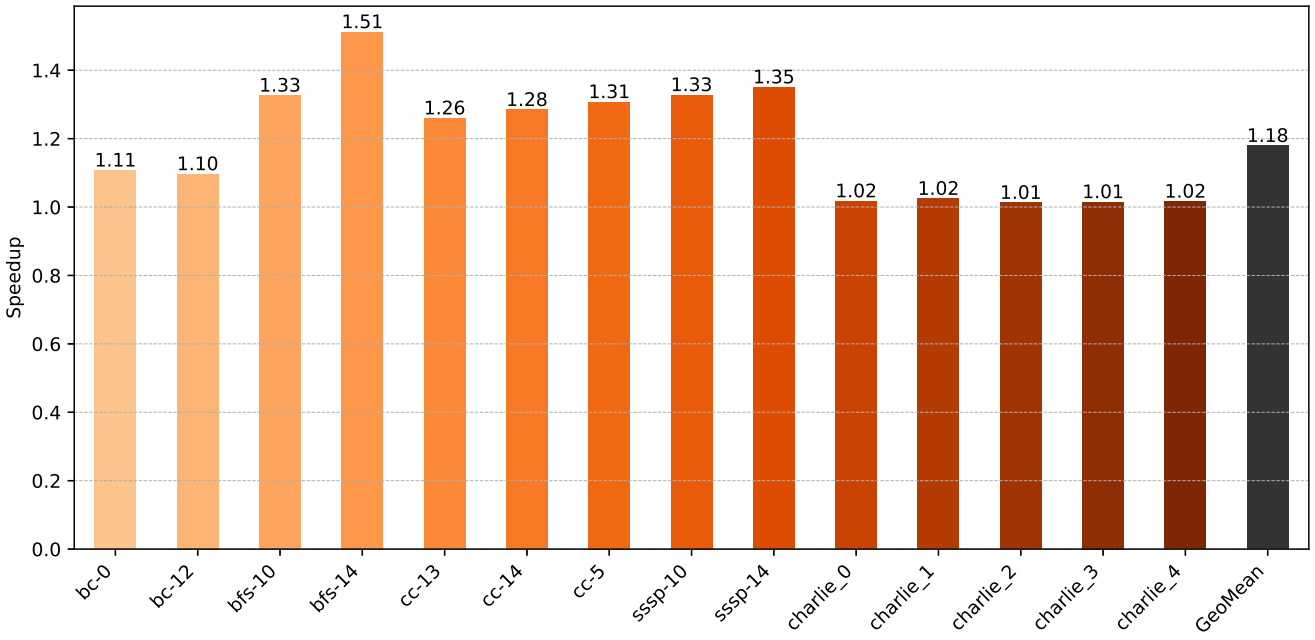

Figure 4: System-aware GHB-stride prefetcher speedup at full bandwidth.

We see higher or equal speedup for all traces except for bfs-10 and bfs-14 where the speedup reduces by 0.03 and 0.1 respectively.

Discussion

From the non-limited bandwidth analysis we gain that the system-aware prefetcher can perform at the same level as the non-system-aware prefetcher. This is an important sanity check to confirm that the system awareness can provide the same optimal performance under optimal conditions. From analysing the limited bandwidth data we understand that the system awareness comes at a cost. We clearly see that for some specific traces we loose a lot of performance due to the system-awareness.

Task 4

In the fourth task we were asked to implement our own prefetcher design. I have opted for a delta correlation prefetcher as it is the natural extension of the stride prefetcher. The basic logic behind a delta correlation prefetcher is to store the deltas, i.e. differences between memory accesses. Over time the prefetcher builds a table associating consecutive deltas. This allows the prefetcher to predict the next delta by computing the last delta and looking in the table which delta is most likely to follow. This design is based on the desing presented in the paper by Grannaes et. al.[2]

Implementation

The delta correlation prefetcher builds on the foundations of the GHB-stride prefetcher. We use the same GHB and IT structs to store the past memory accesses. To this we add a delta-correlation-table(DCT) which stores the different deltas.

```
// delta correlation table entry struct
typedef struct {
    int64_t delta;
    int64_t next_deltas[DCT_NUM_CANDIDATES]; // array of possible following deltas
    uint8_t counters[DCT_NUM_CANDIDATES]; // occurence of following deltas
```

[2]Marius Grannaes, Magnus Jahre, and Lars Natvig. Storage efficient hardware prefetching using Delta-Correlating Prediction Tables. In Journal of Instruction-Level Parallelism, 2011.

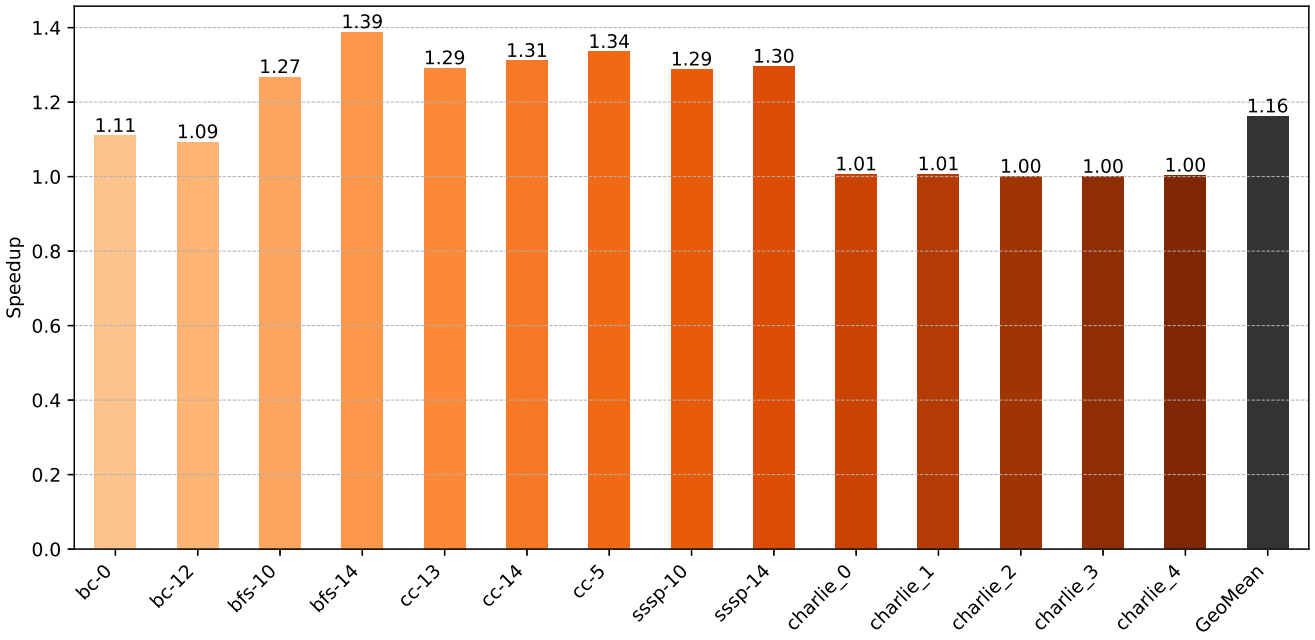

Figure 5: System-aware GHB-stride prefetcher speedup at limited bandwidth.

```
} dct_entry_t;
```

The prefetcher walks the IT and GHB in the same fassion as in the GHB-stride prefetcher. The prefetcher also computes the last two strides, here refered to as `delta1` and `delta2` to avoid confusion, by taking the difference of past cache block addresses. The prefetcher probes the DCT entry of `delta2` with `delta1`. If the entry is already aware of `delta1` then the prefetcher increases the respective counter. If not then the position with the lowest counter is evicted and replaced with `delta1`.

Next the prefetcher tries to predict the future memory access. For this he starts form the known `delta1` and probes its DCT entry for the most likely follow-up delta. It prefetches the cache block address at `current_address + delta`. It then proceeds from the predicted delta and tries to predict further by probing the DCT entry at the location of the predicted delta. The prefetcher repeats this for the same prefetch degree as the stride prefetcher. Allowing it to be system-aware with the same logic.

The prefetcher is based on a set of heuristically determined parameters. The size of the DCT was chosen to be 256 to match the IT and GHB. Each DCT entry holds 4 candidate deltas. The index of the DCT is determined as `(prev_delta + DCT_SIZE/2) & DCT_MAX_ADDR` to avoid mapping deltas with only a sign flip to the same entry.

A further heuristic that is applied is that before prefetching it is checked whether the `delta1` is already known to the entry corresponding to `delta2`. If yes then we prefetch memory and if not then we do not. The idea behind this heuristic is that if the current `delta1` is predicted by `delta2` the chance of being in a predictable pattern is very high. This small distinction has significant impact on the prefetchers performance.

Results

Figure 6 shows the speedup of the delta correlation prefetcher as well as the speedup of the system-aware stride prefetcher relative to the system with no prefetcher at full bandwidth. We can observe clear performance improvement for trace bfs-10 and marginal performance improvements for traces cc-13, cc-14, cc-5, ssp-10, and ssp-14. For traces bc-0 and all charlie traces we see no or minimal performance change and for traces bc-12 and bfs-14 we see clear performance degradation. It is also imporant to note that for the bc-12 trace the speedup is below 1 indicating that the system with the prefetcher performs worse than without. We see a marginal perfromance improvement in the geometric mean.

Figure 7 shows the speedup of the delta correlation prefetcher as well as the speedup of the system-aware stride prefetcher relative to the system with no prefetcher at limited bandwidth. We can see clear performance improvements

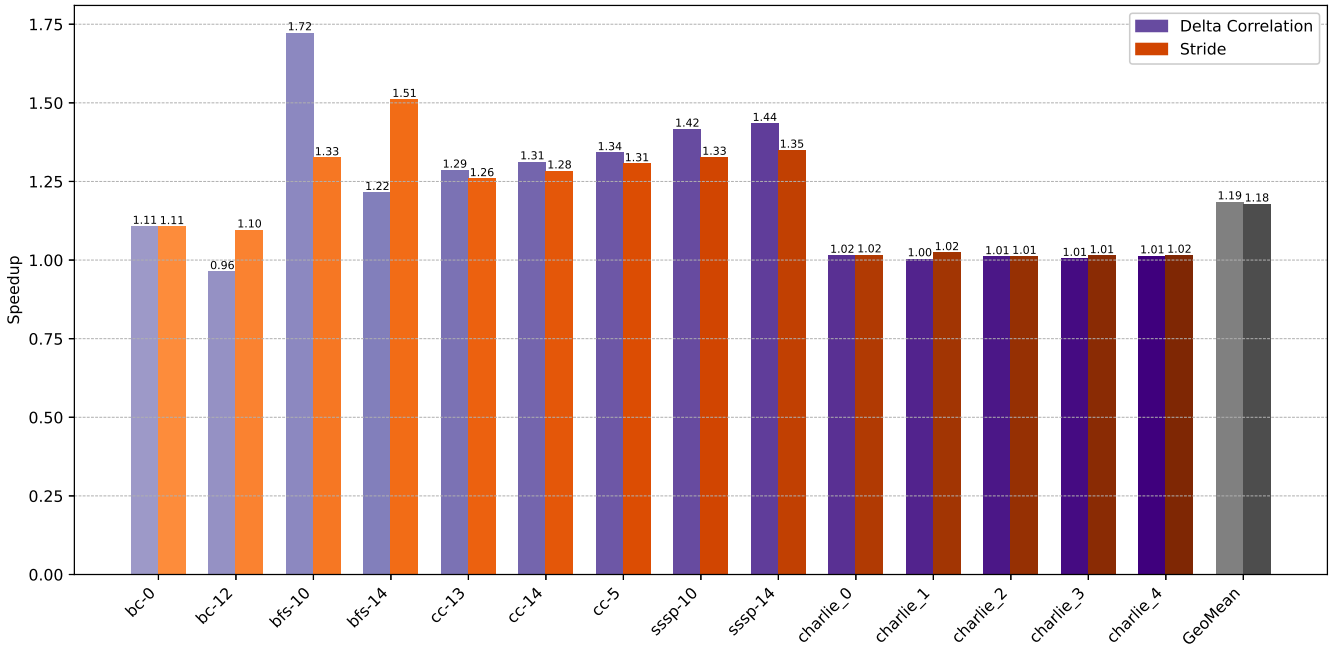

Figure 6: Delta correlation prefetcher and system-aware GHB-stride prefetcher speedup at full bandwidth.

for the traces bfs-10 and bfs-14 as well as marginal performance improvements for cc-13, cc-14, cc-5, sssp-10, and sssp-14. For bc-0 and all charlie traces we see only small or no performanc e difference between the stride and the delta correlation prefetcher. For the trace bc-12 we still see clear performance degradation but now the delta correlation prefetcher has a speedup of 1 indicating that it matches the performance of the baseline system. We see a clear perfromance improvement in the geometric mean.

Discussion

From the data it can clearly be seen that the delta correlation prefetcher performs a lot better in the bandwidth limited system then in the unlimited system. This implies that while it cannot provide a strong peak performance it seems to be very efficient at alocating limited resources. We can also see that there is a very heavy trace dependence in the performance. For certain traces the delta correlation prefetcher clearly outperforms the stride prefetcher, while for others it does not manage to meet the baseline performance. While for some traces the relative performance is consistent between limited and full memory bandwidth for other traces this is not the case. This can be due to the inherent predictability of a trace or specific data patterns used. It could also be that some of these swings could be tuned by changing the ad hoc decided parameterisation of the delta correlation prefetcher. A conclusive reason of these performance swings cannot be given in this discussion as it warants further investigation which would exceed the scope of this lab.

Bonus Task

In the bonus task we are asked to compare our prefetcher to the state-of-the-art pythia prefetcher.[3]

Results

Figure 8 shows the pythia prefetcher compared to both the system-aware stride prefetcher and delta correlation prefetcher relative to the full bandwidth baseline. Pythia performs best on the traces bfs-10 and bfs-14 and performs worst on the traces bc-0 and bc-12. We can also see that pythia provides significant speedup over the baseline for the charlie traces. For the traces cc-13, cc-14, cc-5, sssp-10, and sssp-14 the performance of the three prefetchers is very similar with the delta correlation prefetcher leading in all traces and both pythia and the stride prefetcher in second

[3]Rahul Bera, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning. In MICRO, 2021.

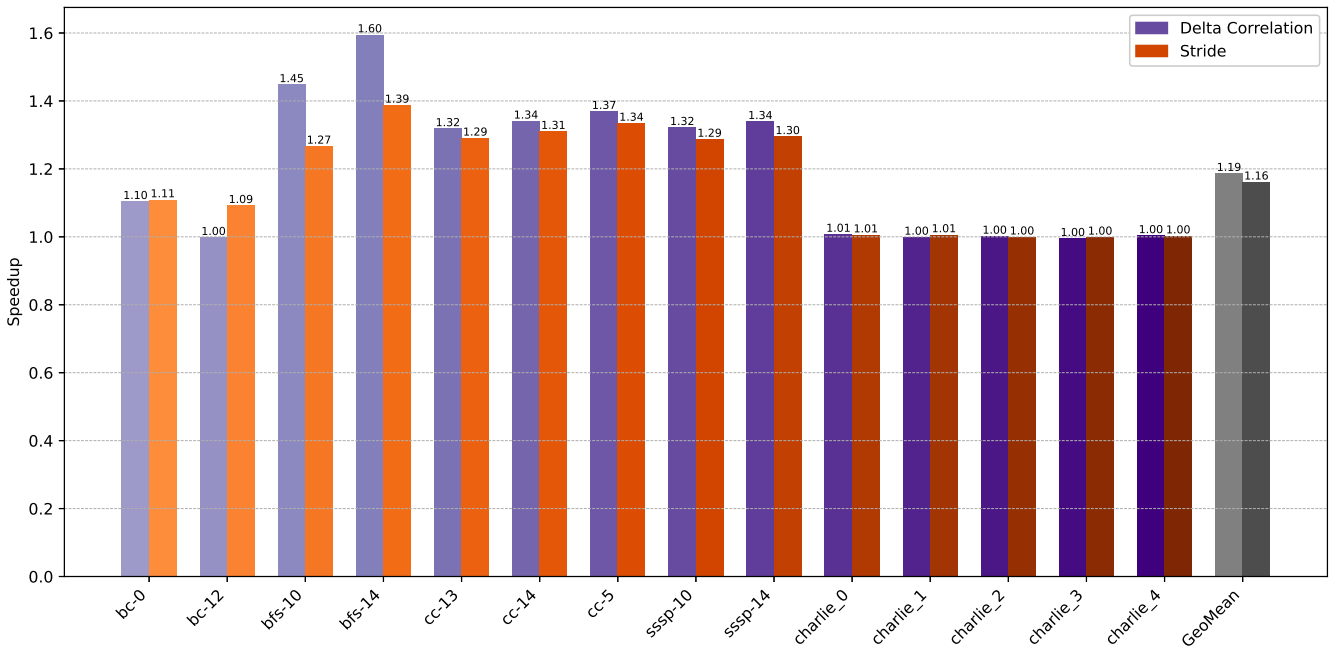

Figure 7: Delta correlation prefetcher and system-aware GHB-stride prefetcher speedup at limited bandwidth.

for some of the traces. We can see that pythia clearly outperforms both the system aware prefetcher and the delta correlation prefetcher in the geometric mean.

Figure 9 shows the pythia prefetcher compared to both the system-aware stride prefetcher and delta correlation prefetcher relative to the limited bandwidth baseline. Pythia still wastly outperforms the other prefetchers in the bfs-10 and bfs-14 traces as well as in the charlie traces. But it shows clear performance degradation in the bc-0 and bc-12 traces where it does not even match the baseline indicated by a speedup below 1. In the traces cc-13, cc-14, cc-5, sssp-10, and sssp-14 it is outperformed by both the stride prefetcher and the delta correlation prefetcher. The geometric mean performance of pythia is now almost equal to that of the delta correlation prefetcher which both outperform the stride prefetcher.

Discussion

Pythia shows over all remarkable performance in the full bandwidth system. And also shows remarkable performance for the bfs traces in the limited bandwidth system. It is also very interesting that pythia is the only prefetcher shown here that achieves a significant speedup for the charlie dataset. From the data we can also conclude that pythia suffers heavily under a memory bandwidth limitation. It is very interesting that the relatively simple delta correlation prefetcher implemented in the scope of this lab almost matche the performance of pythia for the limited bandwidth system.

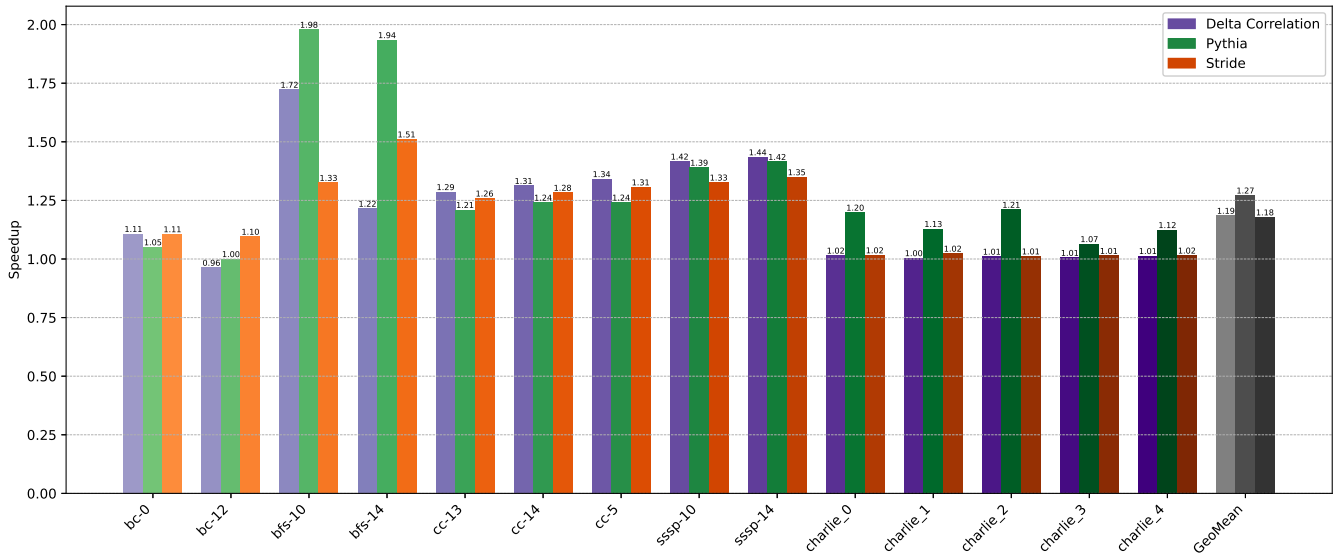

Figure 8: Delta correlation prefetcher, pythia prefetcher, and system-aware GHB-stride prefetcher speedup at full bandwidth.

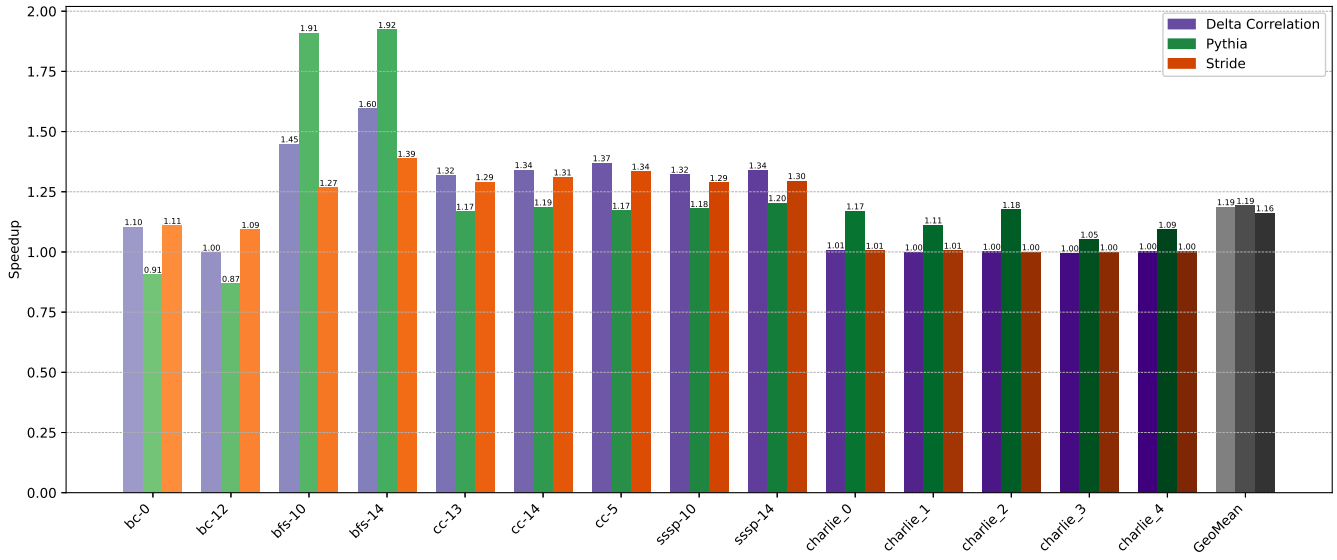

Figure 9: Delta correlation prefetcher, pythia prefetcher, and system-aware GHB-stride prefetcher speedup at limited bandwidth.